

Atelier 8 Cache Redis sous Linux

Fiche étudiant • M2 • Jour 4 • Linux Bash • 1 h 15

Objectif et environnement

Durée : 1 h 15. Vous vérifiez le cache-aside d'une fiche catalogue, sa fraîcheur après modification et le repli lorsque Redis est indisponible. Vous utilisez le produit 42 et la clé shopflow:produit:v1:42. PostgreSQL reste la source de vérité.

L'API contient déjà le cache et l'invalidation. Prouver leur comportement avec les valeurs, le comptage SQL, les durées et les traces. Référence du cours : Jour 4, diapositive 30.

1 Préparation en 10 minutes

Dans .env, définir CACHE_TTL_SECONDS=60. Une minute laisse le temps d'observer une copie ancienne. Conserver LAB_CLIENT_ID=42. Arrêter l'API avec Ctrl+C dans son terminal, puis la redémarrer pour charger cette modification.

```
docker compose up -d --wait
npm start
```

Dans un deuxième terminal Bash placé dans 01_server/api, charger l'aide avec source, vérifier que le TTL affiché vaut 60, puis enregistrer le prix initial. Sinon, corriger .env et redémarrer l'API avant les tests. Garder ce terminal jusqu'à la restauration. Node.js 22 ou supérieur est requis ; aucun PowerShell, jq ou curl n'est nécessaire.

```
source ../../02_Laboratoire/Jour4/Init_Jour4.sh
printf 'TTL configuré : %s secondes\n' "$J4_TTL"
sf_attendre_redis true
docker compose exec -T redis redis-cli DEL shopflow:produit:v1:42
initial=$(sf prix_initial /produits/42)
prixInitial=$(sf_champ "$initial" Body.data.prix)
if [[ "$prixInitial" == '19.90' ]]; then
    prixCible='24.90'
else
    prixCible='19.90'
fi
printf 'Prix initial : %s | Prix cible : %s\n' "$prixInitial" "$prixCible"
```

Noter le prix initial. Le CSV conserve aussi la ligne prix_initial. Tous les prix envoyés par PATCH doivent être des chaînes JSON, par exemple {"prix":"19.90"}. Les modifications concernent uniquement le produit de test.

Paramètre	Valeur de la séance
Produit	42
Clé Redis	shopflow:produit:v1:42
TTL	60 secondes
Prix initial	À relever
Prix cible SQL	19.90 ou 24.90

sf renvoie un objet JSON avec Status, Cache, SqlCount, HttpMs et Traceld. sf_champ extrait un champ, sf_resume affiche les mesures. SqlCount compte les instructions SQL de l'appel HTTP. Exemple : lecture=\$(sf essai /produits/42). En cas d'erreur, arrêter la séquence et vérifier le terminal API avant de poursuivre.

2 Mesure des chemins miss et hit

Temps conseillé : 10 minutes. Supprimer uniquement la clé du produit 42, puis effectuer deux lectures successives avant expiration.

```
docker compose exec -T redis redis-cli DEL shopflow:produit:v1:42
miss=$(sf cache_miss /produits/42)
hit=$(sf cache_hit /produits/42)
sf_resume "$miss" "$hit"
sf_champ "$miss" Body.data
sf_champ "$hit" Body.data
docker compose exec -T redis redis-cli TTL shopflow:produit:v1:42
```

Lecture	HTTP	Cache	SQL attendu
Après DEL	200	miss	1 SELECT
Lecture suivante	200	hit	0

Les contenus data doivent être identiques. Le champ cache diffère. TTL indique entre 0 et 60 secondes si la clé est encore présente. -2 signifie clé absente. -1 indique une clé sans expiration et demande une vérification.

Retrouver les deux TraceId dans le terminal API. Relever durationMs, sqlMs et sqlCount. HttpMs est observé côté client et inclut le coût du client Node.js. Comparer les durées avec le même outil et signaler les premiers appels, parfois plus lents.

Répétition des mesures

Réaliser cinq paires miss / hit, en vidant la clé avant chaque paire.

```
for i in 1 2 3 4 5; do
  docker compose exec -T redis redis-cli DEL shopflow:produit:v1:42 > /dev/null
  m=$(sf "miss_$i" /produits/42)
  h=$(sf "hit_$i" /produits/42)
  sf_resume "$m" "$h"
done
```

Conserver les dix mesures. Expliquer produits.mjs : GET Redis, SELECT sur miss, puis SET avec expiration. Un hit évite le SELECT. Une clé absente ne signifie pas que le produit est absent de la base.

3 Modification SQL et invalidation manuelle

Temps conseillé : 15 minutes. Une écriture SQL directe ne passe pas par updatePrix et ne supprime pas la copie Redis. Exécuter le bloc d'un seul tenant : il recharge le cache, modifie PostgreSQL puis lit immédiatement la copie.

```
docker compose exec -T redis redis-cli DEL shopflow:produit:v1:42
avant=$(sf sql_avant /produits/42)
sqlPrix="UPDATE shopflow.produits SET prix=$prixCible WHERE id=42 RETURNING id,prix;"
docker compose exec -T postgres psql -U cours -d shopflow \
-v ON_ERROR_STOP=1 -c "$sqlPrix"
ancien=$(sf sql_cache_ancien /produits/42)
sf_resume "$ancien"
sf_champ "$ancien" Body.data.prix
```

Attendu : PostgreSQL affiche le prix cible, mais l'API renvoie le prix précédent avec Cache=hit et SqlCount=0 tant que la clé n'a pas expiré. Si elle a expiré, noter le miss et refaire une séquence complète avec un prix de départ différent du prix cible.

Invalider ensuite la copie puis relire la fiche.

```
docker compose exec -T redis redis-cli DEL shopflow:produit:v1:42
nouveau=$(sf sql_apres_del /produits/42)
sf_resume "$nouveau"
sf_champ "$nouveau" Body.data.prix
```

Moment	Prix attendu	Cache	SQL HTTP
Après UPDATE, avant DEL	Prix précédent	hit	0
Après DEL	Prix cible	miss	1 SELECT

L'UPDATE du terminal n'apparaît pas dans X-SQL-Count : il est extérieur à l'appel HTTP. Conserver sa sortie RETURNING avec la réponse ancienne et celle obtenue après DEL.

Analyse de fraîcheur

Expliquer pourquoi le TTL n'actualise pas immédiatement la copie après écriture. Déclarer une fraîcheur tolérée de quelques secondes à une minute pour le catalogue de ce laboratoire. La validation d'un achat exige un contrôle dans la source.

4 Modification par API et invalidation automatique

Temps conseillé : 10 minutes. Le PATCH exécute un UPDATE PostgreSQL, puis supprime la clé Redis après réussite de l'écriture. Avec Redis disponible, la réponse indique invalidation=ok.

```
patch=$(sf patch_prix /produits/42 PATCH '{"prix":"29.90"}')
sf_champ "$patch" Body
apresPatch=$(sf patch_miss /produits/42)
hitPatch=$(sf patch_hit /produits/42)
sf_resume "$patch" "$apresPatch" "$hitPatch"
sf_champ "$apresPatch" Body.data.prix
sf_champ "$hitPatch" Body.data.prix
```

Appel	Résultat attendu	SQL
PATCH	prix 29.90 et invalidation ok	1 UPDATE
Premier GET après PATCH	prix 29.90 et miss	1 SELECT
Deuxième GET	prix 29.90 et hit	0

Le PATCH ne porte pas X-Cache. Son corps indique l'invalidation. Les GET portent X-Cache: miss, hit ou indisponible. Relever les durées et le comptage dans les objets retournés et les traces du serveur.

Ordre des opérations

Dans produits.mjs, retrouver l'UPDATE puis le DEL. L'UPDATE est ici en autocommit : sa réussite valide l'écriture avant invalidation. Avec une transaction explicite, invalider après COMMIT.

Expliquer invalidation=echouee : le prix peut avoir été enregistré dans PostgreSQL alors que la copie Redis reste ancienne. Une erreur d'invalidation ne prouve pas que l'écriture a été annulée.

Preuves à conserver

Joindre le corps du PATCH et les réponses GET. Associer chaque réponse à son Traceld, son prix et son comptage SQL. Comparer au test d'écriture SQL directe de l'étape 3.

5 Expiration et panne de Redis

Temps conseillé : 20 minutes. Attendre 61 secondes après le chargement d'une clé permet de tester son expiration avec le TTL de 60 secondes.

```
docker compose exec -T redis redis-cli DEL shopflow:produit:v1:42
charge=$(sf ttl_chargement /produits/42)
docker compose exec -T redis redis-cli TTL shopflow:produit:v1:42
sleep "$((J4_TTL + 1))"
docker compose exec -T redis redis-cli TTL shopflow:produit:v1:42
expire=$(sf ttl_expire /produits/42)
sf_resume "$expire"
```

Attendu : TTL positif après chargement, -2 après attente, puis Cache=miss avec 1 SELECT. Le prix reste 29.90. Aucun autre GET du produit 42 ne doit intervenir pendant l'attente.

Panne du cache

Arrêter uniquement Redis, attendre la détection par l'API puis relire la fiche. PostgreSQL et l'API doivent rester disponibles.

```
docker compose stop redis
sf_attendre_redis false
panne=$(sf redis_indisponible /produits/42)
sf_resume "$panne"
sf_champ "$panne" Body.data.prix
docker compose start redis
sf_attendre_redis true
```

Scénario	HTTP	Cache	SQL attendu
Après expiration	200	miss	1 SELECT
Redis indisponible	200	indisponible	1 SELECT

Le repli renvoie le prix PostgreSQL, ici 29.90. Conserver trace et durée. Après interruption pendant l'arrêt, redémarrer Redis avant la restauration.

Après le retour de Redis, supprimer la clé puis effectuer deux GET pour confirmer miss / hit. Le repli augmente les lectures PostgreSQL. Expliquer le risque sous forte charge.

6 Restauration et dossier étudiant

Temps conseillé : 10 minutes. Restaurer le prix enregistré au début, avec Redis disponible, même si une manipulation précédente a échoué.

```
docker compose start redis
sf_attendre_redis true
bodyInitial=$(printf '{"prix": "%s"}' "$prixInitial")
retour=$(sf restauration_prix /produits/42 PATCH "$bodyInitial")
controle=$(sf prix_restaurer /produits/42)
if [[ "$(sf_champ "$controle" Body.data.prix)" == "$prixInitial" ]]; then
    echo 'Prix restauré'
else
    echo 'Prix à vérifier'
fi
sf_champ "$retour" Body.invalidation
docker compose exec -T redis redis-cli DEL shopflow:produit:v1:42
```

Attendu : Prix restauré et invalidation=ok. Si le terminal a été fermé, récupérer le prix dans la ligne prix_initial du CSV avant restauration. Ne pas choisir une valeur arbitraire. Remettre le TTL de départ dans .env si demandé, puis redémarrer l'API.

Essai	Prix observé	Cache	SQL	HTTP ms
Miss initial	À relever	miss	1	À relever
Hit suivant	À relever	hit	0	À relever
UPDATE SQL sans DEL	À relever	hit	0	À relever
Après DEL	À relever	miss	1	À relever
GET après PATCH	À relever	miss	1	À relever
Après expiration	À relever	miss	1	À relever
Redis arrêté	À relever	indisponible	1	À relever

Remettre Atelier08_Nom : pattern et clé justifiés, prix initial, CSV, valeurs avant/après, sorties SQL et traces liées par Traceld. Expliquer les durées en distinguant hit, miss et panne.

Course de reconstruction à expliquer

Décrire : A lit un ancien prix sur un miss ; B valide un nouveau prix ; B supprime la clé ; A remet l'ancien prix dans Redis. Expliquer pourquoi DEL après écriture ne supprime pas ce risque et proposer une piste de traitement. Une analyse suffit, sans simulation.

Critères : miss à 1 SELECT, hit à 0 SQL, copie ancienne prouvée après UPDATE direct, nouvelle valeur après invalidation, expiration et repli testés, prix initial restauré. Déclarer la fraîcheur acceptée pour le catalogue et le contrôle transactionnel nécessaire à l'achat.